

Block Verify 与 Entropy Verify 功能设计文档

1. 功能概述

Block Verify 和 Entropy Verify 是 vLLM Ascend 推测解码（Speculative Decoding）拒绝采样流程中的两种优化机制，旨在提升草稿 token 的接受率，从而提高推测解码的整体吞吐量。

1.1 Block Verify（块验证）

Block Verify 将同一请求的所有草稿 token 作为一个整体进行验证，使用累积概率乘积（cumulative probability product）来判断接受区间，而非逐 token 独立判断。该机制源自 MagicMTP 思想，在 `max_spec_len >= 3` 时效果显著。

核心思想：传统拒绝采样中，每个 token 独立生成均匀随机数 u_i 并判断 $p_{\text{target}}(x_i)/p_{\text{draft}}(x_i) \geq u_i$ ，一旦某个 token 被拒绝则后续 token 全部丢弃。Block Verify 改为计算累积接受概率 $\pi = \prod \min(p_{\text{target}}(x_i)/p_{\text{draft}}(x_i), 1)$ 和累积均匀概率 $U = \prod u_i$ ，然后找到最大的 k 使得 $\pi_k \geq U_k$ ，接受前 k 个 token。

1.2 Entropy Verify（熵验证）

Entropy Verify 根据目标模型分布的熵值动态调整接受阈值。高熵（不确定分布）时降低阈值使更多 token 被接受，低熵（确定分布）时保持较严格的阈值。

核心思想：计算目标分布的熵 $H = -\sum p(x) \log p(x)$ ，然后计算调整后的阈值 $\text{threshold} = \min(\exp(-H \times \alpha), \text{posterior_threshold})$ ，用 $\text{threshold} \times U$ 替代原始均匀随机数 U 作为接受判据。

1.3 组合使用

两者可独立启用，也可组合使用。组合时，Block Verify 的累积接受概率与 Entropy Verify 的熵调整阈值同时生效，接受条件变为 $\pi_k \geq \text{threshold}_k \times U_k$ 。

2. 核心算法原理

2.1 传统拒绝采样算法

对于请求中的第 i 个草稿 token x_i ：

接受条件： $p_{\text{target}}(x_i) / p_{\text{draft}}(x_i) \geq u_i$
其中 $u_i \sim \text{Uniform}(0, 1)$

若被拒绝，从修正分布 $\max(p_{\text{target}} - p_{\text{draft}}, 0) / q$ 中采样恢复 token，其中 q 为指数分布归一化因子。

2.2 Block Verify 算法

对于请求中所有草稿 token $[x_0, x_1, \dots, x_{n-1}]$ ：

1. 初始化： $\pi = 1.0$, $U = 1.0$, $\text{last_accepted_pos} = -1$
2. 对每个 token 位置 $\text{pos} = 0, 1, \dots, n-1$:
 - a. 计算 $\text{ratio} = \min(p_{\text{target}}(x_{\text{pos}}) / p_{\text{draft}}(x_{\text{pos}}), 1.0)$
 - b. $\pi = \pi \times \text{ratio}$
 - c. $U = U \times u_{\text{pos}}$ (u_{pos} 为该位置的均匀随机数)
 - d. 若 $\text{draft_prob} > 0$ 且 $\pi \geq U$:
 $\text{last_accepted_pos} = \text{pos}$
3. 接受 $[0, \text{last_accepted_pos}]$ 范围内的所有草稿 token
4. 若 $\text{last_accepted_pos} + 1 < n$:
在 $\text{last_accepted_pos} + 1$ 处放置恢复 token
5. 否则：
在末尾放置 bonus token

关键差异：传统方法在第一个拒绝处立即停止；Block Verify 遍历所有位置计算累积概率，即使中间位置的独立比率较低，只要累积乘积满足条件，后续 token 仍可能被接受。

2.3 Entropy Verify 算法

1. 计算目标分布的熵:

$$H = -\sum_{v \in V} p(v) \times \log(p(v) + \epsilon)$$

2. 计算熵调整阈值:

$$\text{threshold} = \min(\exp(-H \times \alpha), \text{posterior_threshold})$$

3. 修改接受条件:

$$\text{传统: } p_{\text{target}}(x_i) / p_{\text{draft}}(x_i) \geq u_i$$

$$\text{熵验证: } p_{\text{target}}(x_i) / p_{\text{draft}}(x_i) \geq \text{threshold} \times u_i$$

参数说明:

- `posterior_threshold` (默认 0.95): 阈值上限, 即使熵极低, 有效阈值也不超过此值
- `posterior_alpha` (默认 0.4): 熵缩放因子, 控制熵对阈值的影响强度
- 当 $\alpha = 0$ 时, $\text{threshold} = \min(1, \text{posterior_threshold}) = \text{posterior_threshold}$, 熵无影响
- 当 α 较大时, 高熵 token 的阈值显著降低, 接受率大幅提升

2.4 Block Verify + Entropy Verify 组合算法

1. 初始化: $\pi = 1.0$, $U = 1.0$, $\text{last_accepted_pos} = -1$

2. 对每个 token 位置 $\text{pos} = 0, 1, \dots, n-1$:

- a. 计算 $\text{ratio} = \min(p_{\text{target}}(x_{\text{pos}}) / p_{\text{draft}}(x_{\text{pos}}), 1.0)$

- b. $\pi = \pi \times \text{ratio}$

- c. $U = U \times u_{\text{pos}}$

- d. 计算该位置的熵 H_{pos} 及阈值 $\text{threshold}_{\text{pos}}$

- e. 若 $\text{draft_prob} > 0$ 且 $\pi \geq \text{threshold}_{\text{pos}} \times U$:

$$\text{last_accepted_pos} = \text{pos}$$

3. 后续处理同 Block Verify

2.5 恢复 Token 采样的差异

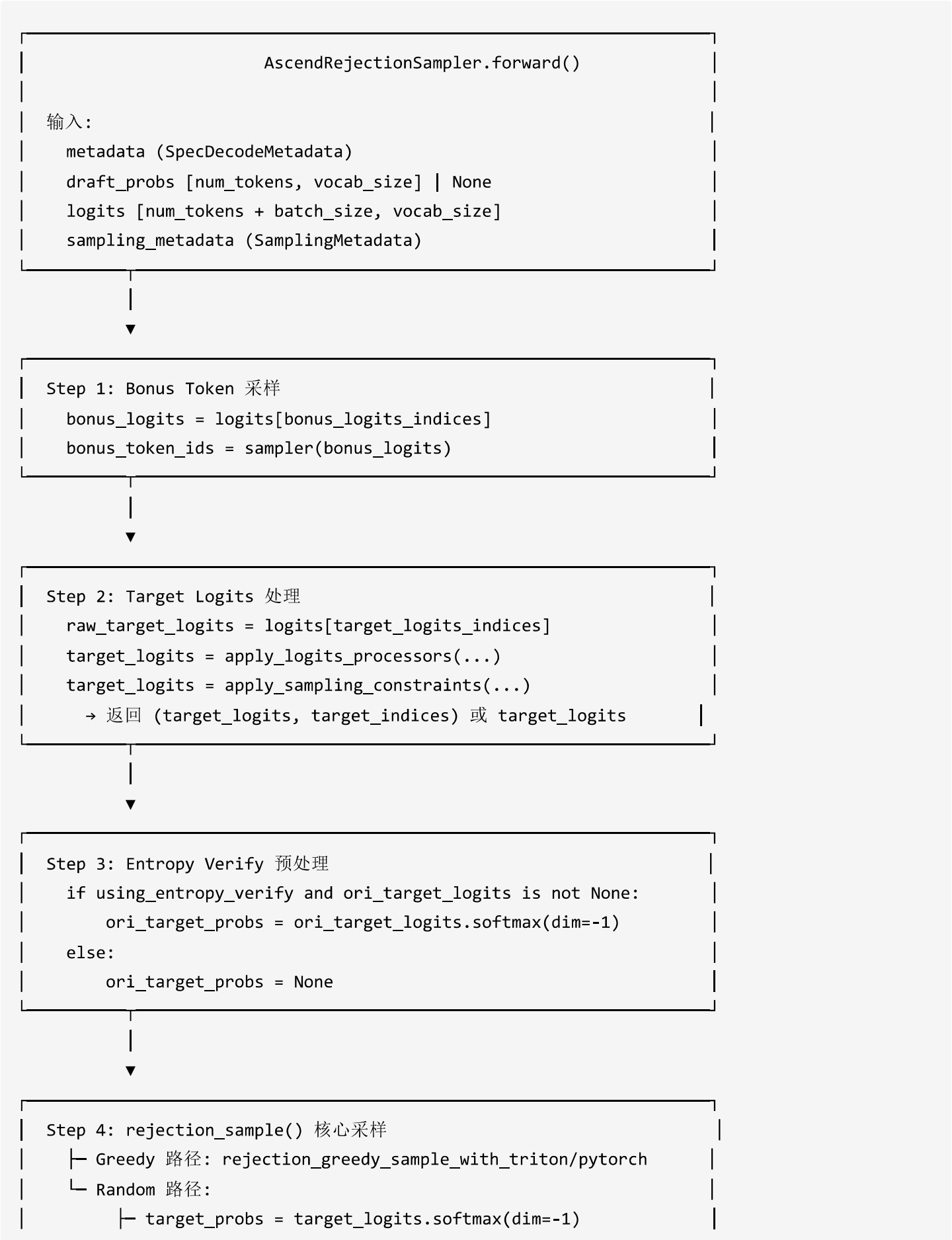
Block Verify 模式下, 恢复 token 的采样分布与传统模式不同:

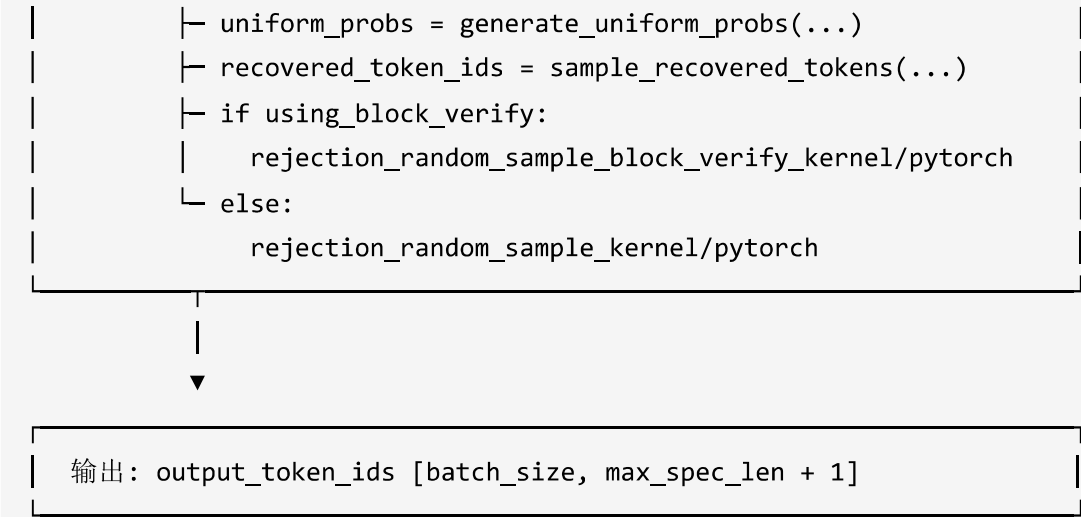
- **传统模式:** $\text{residual} = \max(p_{\text{target}} - p_{\text{draft}}, 0)$
- **Block Verify 模式:** $\text{residual} = \max(\pi_i \times p_{\text{target}} - p_{\text{draft}}, 0)$

其中 π_i 是到第 i 个位置的累积接受概率。这确保了恢复 token 的分布正确反映了条件概率。

3. 数据流程

3.1 整体数据流





3.2 关键数据结构

数据	形状	说明
draft_token_ids	[num_tokens]	草稿模型生成的 token ID
draft_probs	[num_tokens, vocab_size]	草稿模型概率分布 (NGRAM 模式下为 None)
target_probs	[num_tokens, vocab_size] 或 [num_tokens, selected_vocab_size]	目标模型概率分布
target_indices	[num_tokens, selected_vocab_size]	Reduce Sampling 下的全局词表索引
bonus_token_ids	[batch_size, 1]	Bonus token ID
recovered_token_ids	[num_tokens]	恢复 token ID
uniform_probs	[num_tokens]	均匀随机数
ori_target_probs	[num_tokens, vocab_size]	原始目标概率 (用于熵计算)
cu_num_draft_tokens	[batch_size]	累积草稿 token 数
is_greedy	[batch_size]	是否贪心采样
output_token_ids	[batch_size, max_spec_len + 1]	输出 token ID

4. 接口定义

4.1 配置接口 — RejectionSamplerConfig

```
class RejectionSamplerConfig:
    def __init__(self, config: dict | None = None):
        self.enable_block_verify: bool = config.get("enable_block_verify", False)
        self.enable_entropy_verify: bool = config.get("enable_entropy_verify", False)
        self.posterior_threshold: float = config.get("posterior_threshold", 0.95)
        self.posterior_alpha: float = config.get("posterior_alpha", 0.4)
        self._validate()
```

配置参数：

参数	类型	默认值	约束	说明
enable_block_verify	bool	False	—	是否启用块验证
enable_entropy_verify	bool	False	—	是否启用熵验证
posterior_threshold	float	0.95	(0, 1]	熵调整阈值上限
posterior_alpha	float	0.4	>= 0	熵缩放因子

使用方式：

```
# 在线推理
vllm serve <model> --additional-config \
    '{"rejection_sampler_config": {"enable_block_verify": true, \
    "enable_entropy_verify": true, "posterior_threshold": 0.95, \
    "posterior_alpha": 0.4}}'
```

```
# 离线推理
llm = LLM(model, additional_config={
    "rejection_sampler_config": {
        "enable_block_verify": True,
        "enable_entropy_verify": True,
        "posterior_threshold": 0.95,
        "posterior_alpha": 0.4,
    }
})
```

4.2 核心采样接口 — rejection_sample()

```
def rejection_sample(
    draft_token_ids: torch.Tensor,          # [num_tokens]
    num_draft_tokens: list[int],            # [batch_size]
    max_spec_len: int,
    cu_num_draft_tokens: torch.Tensor,      # [batch_size]
    draft_probs: torch.Tensor | None,      # [num_tokens, vocab_size]
    target_logits_or_tuple: torch.Tensor | tuple[torch.Tensor, torch.Tensor | None],
    bonus_token_ids: torch.Tensor,         # [batch_size, 1]
    sampling_metadata: SamplingMetadata,
    ori_target_logits: torch.Tensor | None = None, # [num_tokens, vocab_size]
) -> torch.Tensor:                        # [batch_size, max_spec_len + 1]
```

4.3 Triton Kernel 接口

rejection_random_sample_kernel

标准拒绝采样 Triton kernel，支持 Entropy Verify：

```
@triton.jit
def rejection_random_sample_kernel(
    output_token_ids_ptr, cu_num_draft_tokens_ptr,
    draft_token_ids_ptr, draft_probs_ptr,
    target_probs_ptr, target_indices_ptr,
    bonus_token_ids_ptr, recovered_token_ids_ptr,
    uniform_probs_ptr, is_greedy_ptr,
    max_spec_len, vocab_size, global_vocab_size, vec_len,
    ori_target_probs_ptr,
    NO_ORI_TARGET_PROBS: tl.constexpr,
    NO_DRAFT_PROBS: tl.constexpr,
    ENABLE_REDUCE_SAMPLING: tl.constexpr,
    ENTROPY_VERIFY: tl.constexpr,
    BLOCK_SIZE: tl.constexpr,
    VOCAB_BLOCK_SIZE: tl.constexpr = 512,
    POSTERIOR_THRESHOLD: tl.constexpr = 0.95,
    POSTERIOR_ALPHA: tl.constexpr = 0.4,
    SUB_BLOCK: tl.constexpr = 4096,
    EPSILON: tl.constexpr = 1e-10,
)
```

rejection_random_sample_block_verify_kernel

Block Verify 拒绝采样 Triton kernel，同时支持 Entropy Verify：

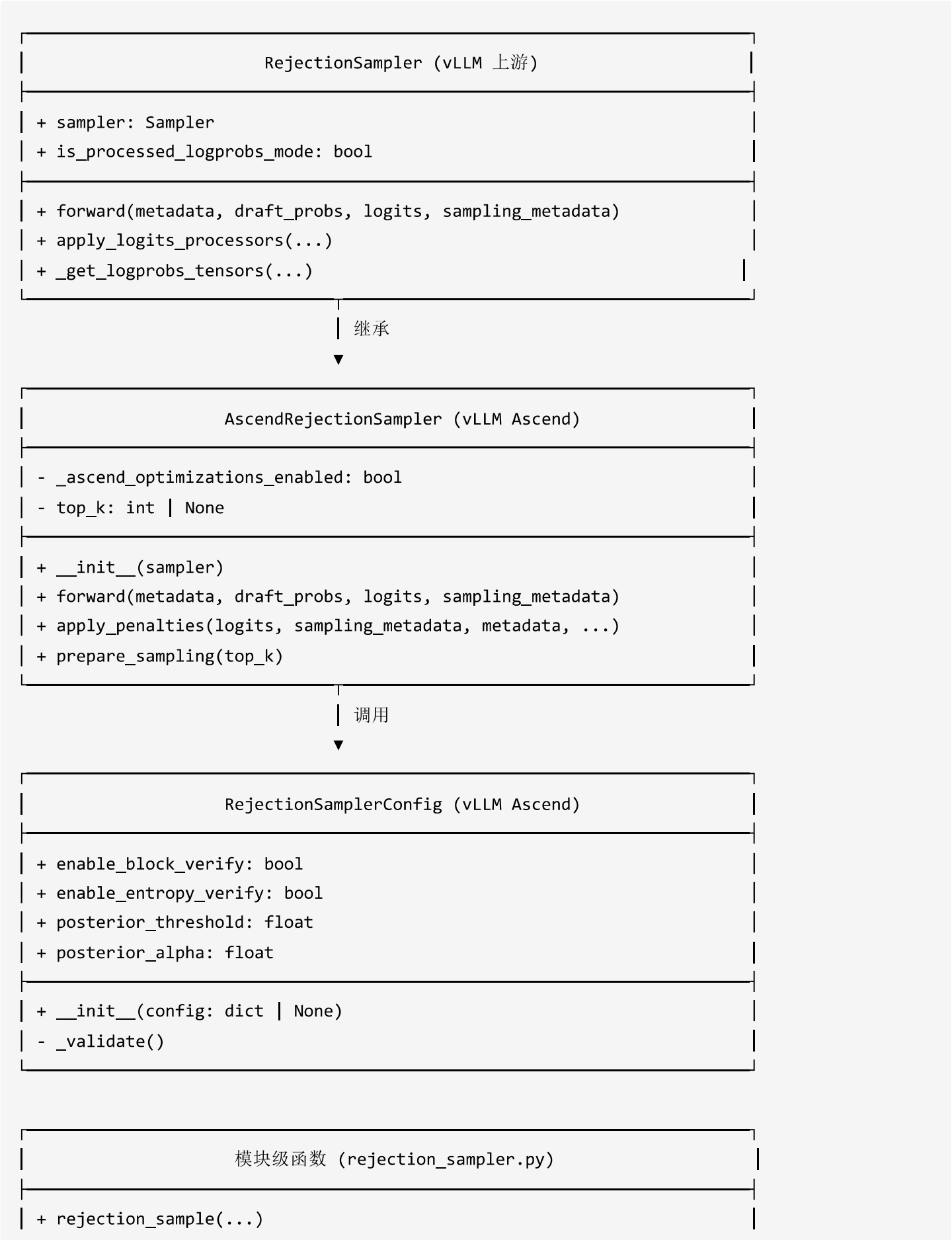
```
@triton.jit
def rejection_random_sample_block_verify_kernel(
    output_token_ids_ptr, cu_num_draft_tokens_ptr,
    draft_token_ids_ptr, draft_probs_ptr,
    target_probs_ptr, target_indices_ptr,
    bonus_token_ids_ptr, recovered_token_ids_ptr,
    uniform_probs_ptr, is_greedy_ptr,
    max_spec_len, vocab_size, global_vocab_size, vec_len,
    ori_target_probs_ptr,
    NO_ORI_TARGET_PROBS: tl.constexpr,
    NO_DRAFT_PROBS: tl.constexpr,
    ENABLE_REDUCE_SAMPLING: tl.constexpr,
    BLOCK_SIZE: tl.constexpr,
    ENTROPY_VERIFY: tl.constexpr,
    VOCAB_BLOCK_SIZE: tl.constexpr = 512,
    POSTERIOR_THRESHOLD: tl.constexpr = 0.95,
    POSTERIOR_ALPHA: tl.constexpr = 0.4,
    SUB_BLOCK: tl.constexpr = 4096,
    EPSILON: tl.constexpr = 1e-10,
)
```

4.4 PyTorch 回退接口

函数	说明
<code>rejection_random_sample_pytorch()</code>	标准拒绝采样 PyTorch 实现，支持 Entropy Verify
<code>rejection_random_sample_block_verify_pytorch()</code>	Block Verify 拒绝采样 PyTorch 实现，支持 Entropy Verify
<code>sample_recovered_tokens_pytorch()</code>	标准恢复 token 采样
<code>sample_recovered_tokens_blockwise_pytorch()</code>	Block Verify 模式下的恢复 token 采样

5. 类结构设计

5.1 类图



```

| + greedy_sample(logits)
| + apply_sampling_constraints(logits, cu_num_draft_tokens, ...)
| + expand_batch_to_tokens(x, cu_num_tokens, num_tokens, ...)
| + sample_recovered_tokens(...)
| - rejection_greedy_sample_spec_len_1_pytorch(...)
| - rejection_greedy_sample_pytorch(...)
| - rejection_random_sample_pytorch(...)
| - rejection_random_sample_block_verify_pytorch(...)
| - sample_recovered_tokens_pytorch(...)
| - sample_recovered_tokens_blockwise_pytorch(...)
| - expand_pytorch(...)

```

Triton Kernels (reject_sample.py)

```

| + rejection_greedy_sample_spec_len_1_triton(...)
| + rejection_greedy_sample_triton(...)
| + rejection_random_sample_kernel(...)
| + rejection_random_sample_block_verify_kernel(...)
| + sample_recovered_tokens_kernel(...)
| + expand_kernel(...)
| + cal_grid_and_block_size(batch_size)
| + rejection_greedy_sample_with_triton(...)
| + expand_triton(...)

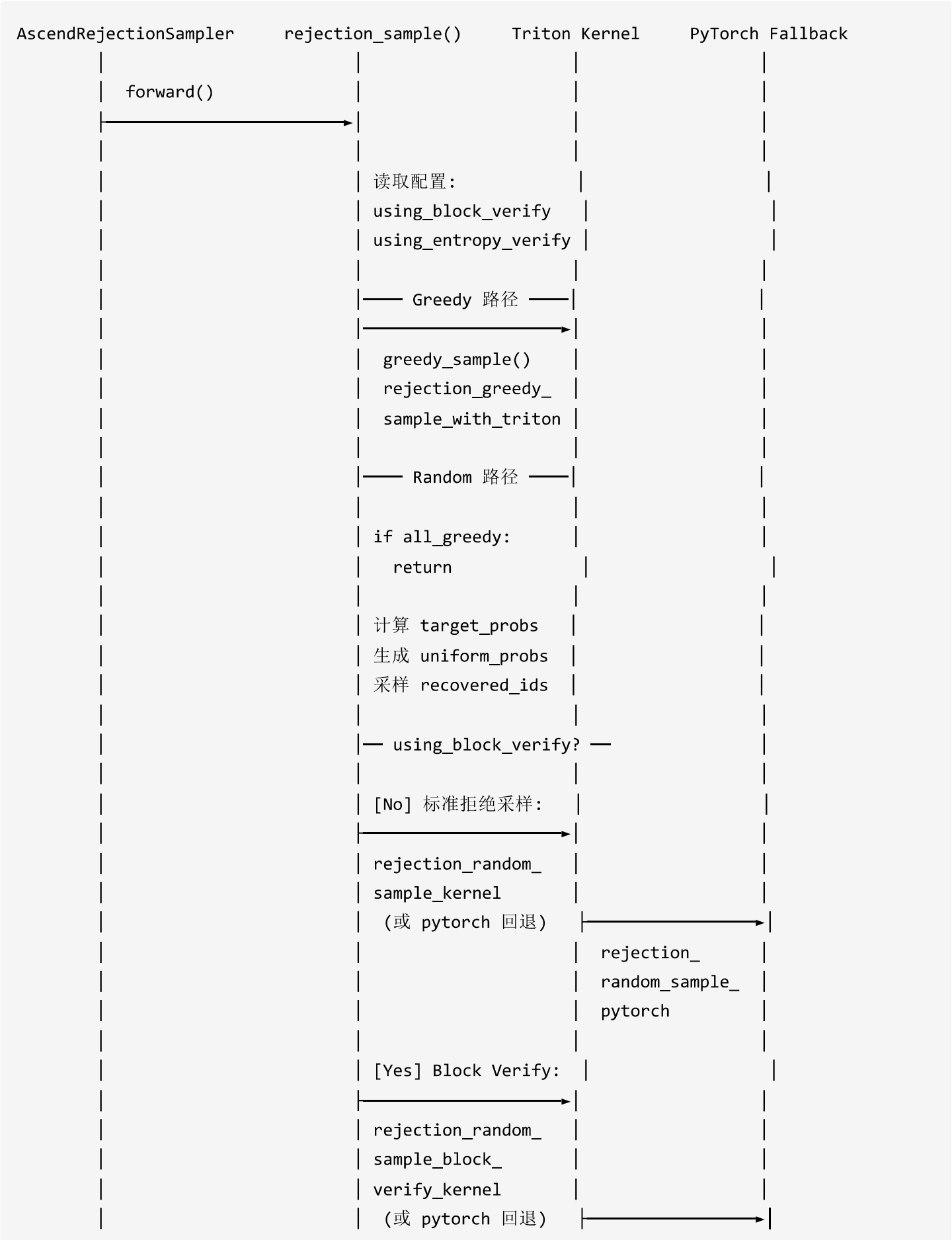
```

5.2 类关系说明

- `AscendRejectionSampler` 继承自 vLLM 上游的 `RejectionSampler`，重写了 `forward()`、`apply_penalties()` 等方法以适配 Ascend NPU
- `RejectionSamplerConfig` 作为独立配置类，通过 `get_ascend_config().rejection_sampler_config` 获取
- 模块级函数 `rejection_sample()` 是核心调度入口，根据配置选择不同的采样路径
- Triton Kernel 提供高性能 NPU 实现，PyTorch 函数作为回退

6. 关键流程实现

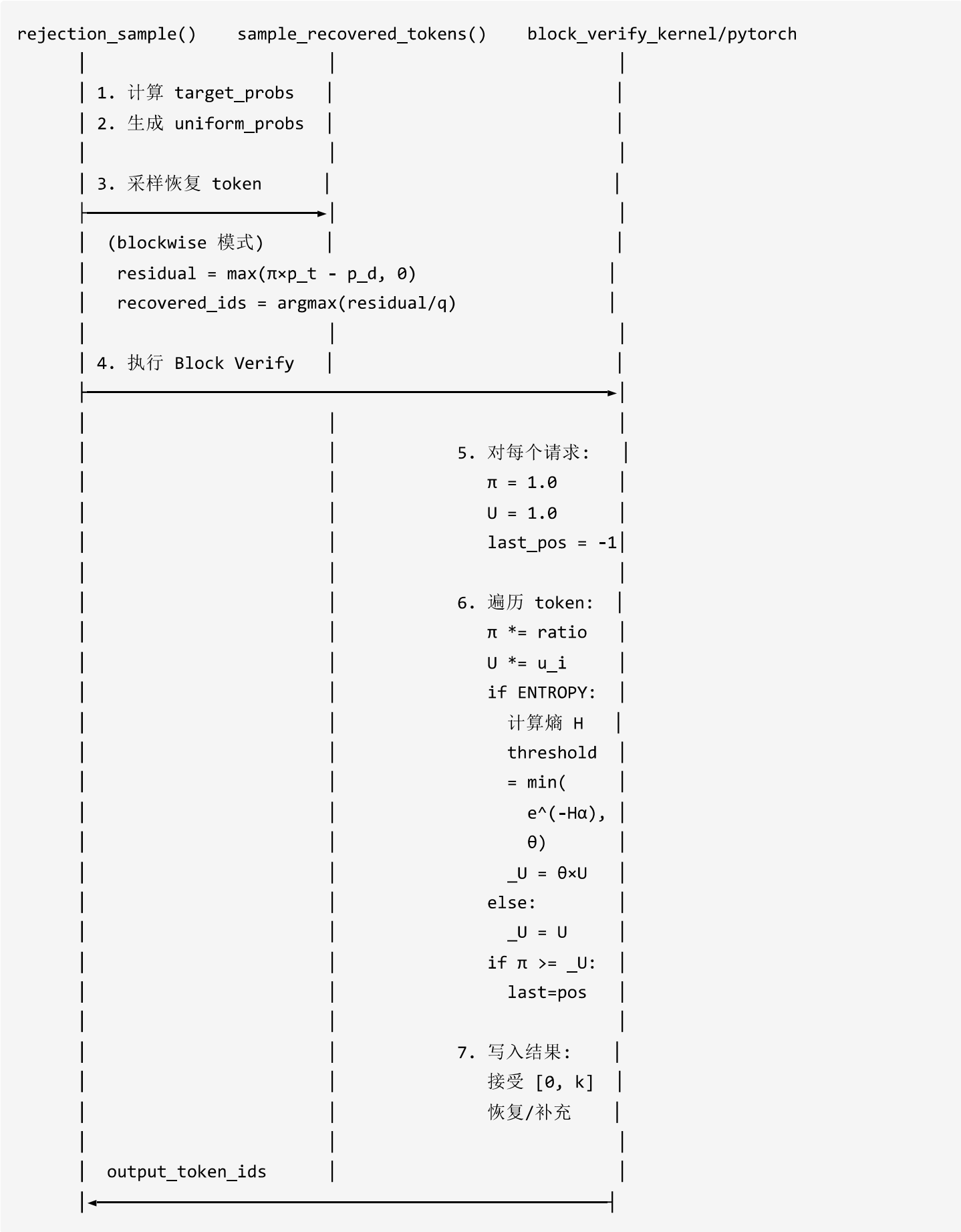
6.1 拒绝采样路径选择时序图



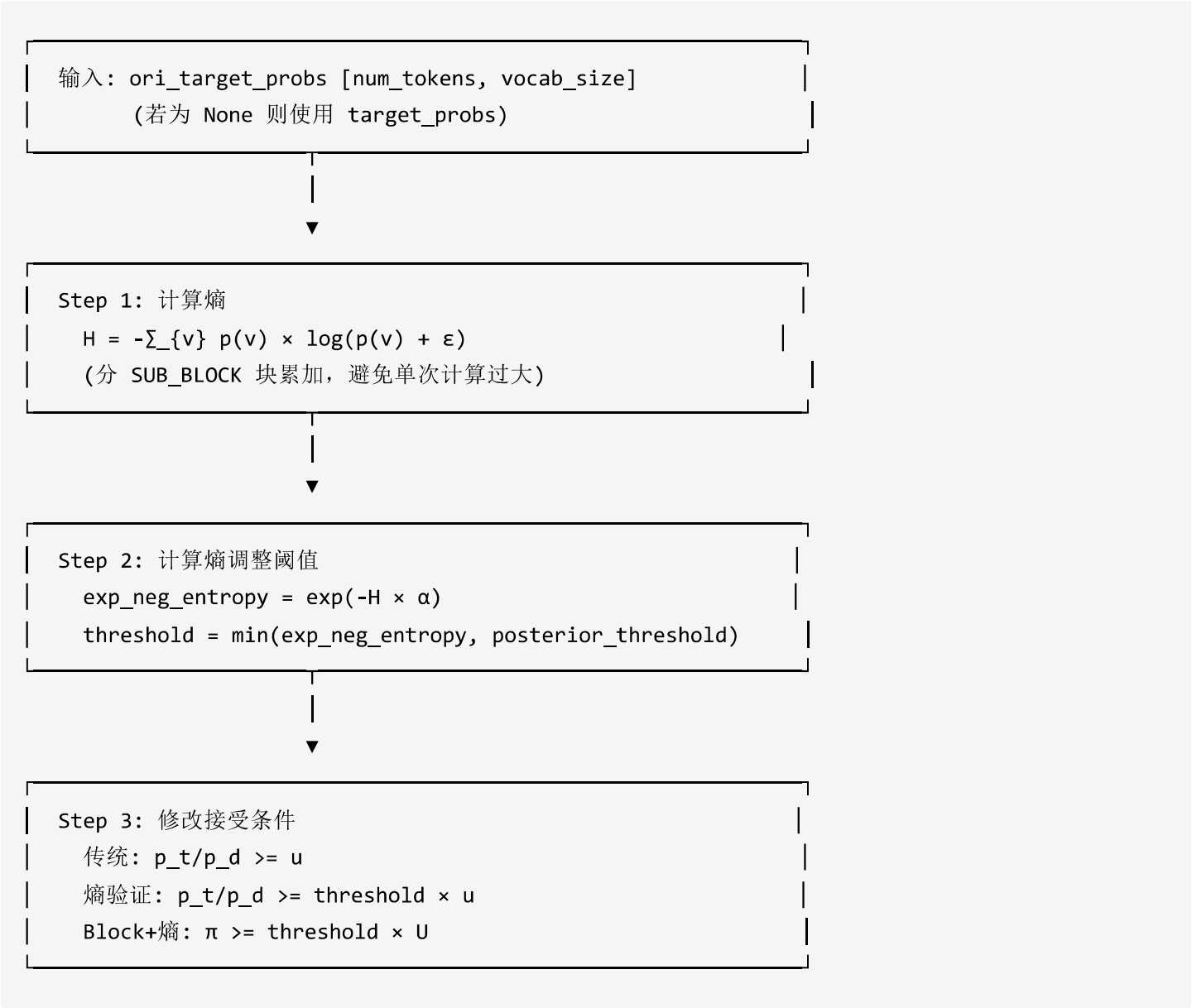
```
|  
|  
|  
|  
|  
| return output_token_ids |  
|←────────────────────────|
```

```
| rejection_ |  
| random_sample_ |  
| block_verify_ |  
| pytorch |  
|  
|  
|
```

6.2 Block Verify 核心流程时序图



6.3 Entropy Verify 阈值计算流程



7. 性能优化策略

7.1 Triton Kernel 加速

- **自定义 Kernel**: 所有核心计算（拒绝采样、恢复 token 采样、扩展操作）均提供 Triton 实现，直接在 NPU Vector Core 上执行
- **Grid/Block 自适应**: `cal_grid_and_block_size()` 根据 batch_size 和 vectorcore_num 自动调整并行度
- **constexpr 特化**: `NO_DRAFT_PROBS`、`ENABLE_REDUCE_SAMPLING`、`ENTROPY_VERIFY` 等编译时常量避免运行时分支

7.2 熵计算优化

- **SUB_BLOCK 分块**: 熵计算以 4096 为子块大小分块累加，避免单次分配过大的共享内存
- **ori_target_probs 复用**: 优先使用原始 logits 的 softmax 结果计算熵，避免经过 top-k/top-p 截断后熵值失真
- **Fallback 机制**: 当 `ori_target_probs` 不可用时，回退使用 `target_probs` 计算熵

7.3 内存优化

- **原地操作**: `logits.div_(temperature)` 等原地运算减少内存分配
- **pin_memory**: CPU 端常量张量使用 `pin_memory=True` 加速 CPU→NPU 传输
- **non_blocking 传输**: `to(device, non_blocking=True)` 实现异步数据搬运

8. 错误处理机制

8.1 配置验证

`RejectionSamplerConfig._validate()` 在初始化时进行严格校验:

校验项	条件	异常信息
<code>enable_block_verify</code> 类型	<code>isinstance(bool)</code>	<code>must be a bool, got {type}</code>
<code>enable_entropy_verify</code> 类型	<code>isinstance(bool)</code>	<code>must be a bool, got {type}</code>
<code>posterior_threshold</code> 类型	<code>isinstance(int/float)</code>	<code>must be a float, got {type}</code>
<code>posterior_alpha</code> 类型	<code>isinstance(int/float)</code>	<code>must be a float, got {type}</code>
<code>posterior_threshold</code> 范围	<code>0 < val <= 1</code>	<code>must be in (0, 1], got {val}</code>
<code>posterior_alpha</code> 范围	<code>val >= 0</code>	<code>must be >= 0, got {val}</code>

8.2 运行时断言

```
assert metadata.max_spec_len <= MAX_SPEC_LEN           # 最大推测长度限制
assert draft_token_ids.ndim == 1                        # 维度校验
assert draft_probs is None or draft_probs.ndim == 2    # 维度校验
assert cu_num_draft_tokens.ndim == 1                   # 维度校验
assert target_logits.ndim == 2                        # 维度校验
assert target_logits.shape[0] == num_tokens            # token 数一致性
assert draft_token_ids.is_contiguous()                 # 内存连续性
assert target_probs.is_contiguous()                   # 内存连续性
```

8.3 数值安全

- **EPSILON = 1e-10**: `log(probs + EPSILON)` 防止 `log(0)` 产生 `-inf`
- **draft_prob > 0 检查**: 避免 `target_prob / 0` 产生 NaN, draft_prob 为 0 时直接拒绝
- **q 值安全处理**: `q_values_safe = where(q == 0 | isinf(q), epsilon, q)` 防止除零
- **ratio 上限**: `min(ratio, 1.0)` 确保累积概率不超过 1

8.4 回退机制

- **Triton 不可用**: 当 `HAS_TRITON = False` 时, 自动回退到 PyTorch 实现
- **Reduce Sampling 不可用**: 当 `target_indices` 为 None 时, 回退到全词表模式
- **ori_target_probs 不可用**: 当 `ori_target_probs` 为 None 时, 使用 `target_probs` 计算熵
- **Fallback 路径警告**: 非 Reduce Sampling 路径会输出 `logger.warning_once`

8.5 Block Verify 前置条件

```
using_block_verify = max_spec_len >= 3 and bool(
    get_ascend_config().rejection_sampler_config.enable_block_verify
)
```

即使配置启用了 Block Verify, 当 `max_spec_len < 3` 时仍自动降级为标准模式。